CS 301
High-Performance Computing

Assignment 08 – Parallel Interpolation with Particle Mover using MPI

Visha Sitapara (202301414)
Dhruv Patel (202301095)

April 29, 2026

Contents

1 Introduction **3**

2 Hardware Details **3**
2.1 Hardware Details for LAB207 PCs 3
2.2 Hardware Details for HPC Cluster (Compute Nodes gies1-4) 4

3 Problem Description **5**
3.1 Linear Interpolation (Scatter) 5
3.2 Particle Update using Reverse Interpolation (Mover) 5
3.3 Parallelization Strategies: 6

4 Benchmarking Methodology **6**

5 Graphical Results **7**
5.1 Speedup vs Cores 7
5.1.1 Config01: $N_x = 250$, $N_y = 100$, Number of points $= 0.9 \times 10^6$, Maxiter $= 10$. 7
5.1.2 Config02: $N_x = 250$, $N_y = 100$, Number of points $= 5 \times 10^6$, Maxiter $= 10$ . . 8
5.1.3 Config03: $N_x = 500$, $N_y = 200$, Number of points $= 3.6 \times 10^6$, Maxiter $= 10$. 9
5.1.4 Config04: $N_x = 500$, $N_y = 200$, Number of points $= 20 \times 10^6$, Maxiter $= 10$. 10
5.1.5 Config05: $N_x = 1000$, $N_y = 400$, Number of points $= 14 \times 10^6$, Maxiter $= 10$ 11
5.2 Execution time vs Cores 12
5.2.1 Config01: $N_x = 250$, $N_y = 100$, Number of points $= 0.9 \times 10^6$, Maxiter $= 10$. 12
5.2.2 Config02: $N_x = 250$, $N_y = 100$, Number of points $= 5 \times 10^6$, Maxiter $= 10$ . . 13
5.2.3 Config03: $N_x = 500$, $N_y = 200$, Number of points $= 3.6 \times 10^6$, Maxiter $= 10$. 14
5.2.4 Config04: $N_x = 500$, $N_y = 200$, Number of points $= 20 \times 10^6$, Maxiter $= 10$. 15
5.2.5 Config05: $N_x = 1000$, $N_y = 400$, Number of points $= 14 \times 10^6$, Maxiter $= 10$ 16

6 Analysis and Questions **17**

7 Conclusions **21**

1 Introduction

This lab focuses on the parallelization of the complete particle-mesh workflow, including both the scatter point-to-mesh interpolation and the reverse interpolation (gather: mesh-to-point) acting as a particle mover. In the current assignment, the focus is on implementing the same overall workflow using a hybrid approach with MPI + OpenMP. The experiment highlights the contrasting challenges of handling race conditions across distributed nodes during the scatter phase versus optimizing memory-bound operations during the gather phase. The MPI layer enables the use of all cores available across multiple compute nodes on the HPC.

2 Hardware Details

2.1 Hardware Details for LAB207 PCs

- Architecture: x86_64
- CPU op-mode(s): 32-bit, 64-bit
- Address sizes: 46 bits physical, 48 bits virtual
- Byte Order: Little Endian
- CPU(s): 12
- On-line CPU(s) list: 0-11
- Thread(s) per core: 2
- Core(s) per socket: 6
- Socket(s): 1
- NUMA node(s): 1
- Vendor ID: GenuineIntel
- CPU family: 6
- Model: 151
- Model name: 12th Gen Intel(R) Core(TM) i5-12500
- Stepping: 5
- CPU max MHz: 4600.0000
- CPU min MHz: 800.0000
- BogoMIPS: 5990.40
- Virtualization: VT-x
- L1d cache: 288K

- L1i cache: 192K
- L2 cache: 7.5M
- L3 cache: 18M
- NUMA node0 CPU(s): 0-11
- Flags: fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe syscall nx pdpe1gb rdtscp lm constant_tsc arch_perfmon pebs bts rep_good nopl xtopology nonstop_tsc aperfmperf eagerfpu pni pclmulqdq dtes64 monitor ds_cpl vmx smx est tm2 ssse3 fma cx16 xtpr pdcm pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand lahf_lm abm epb invpcid_single tpr_shadow vnmi flexpriority ept vpid fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid xsaveopt dtherm ida arat pln pts

2.2 Hardware Details for HPC Cluster (Compute Nodes gies1-4)

- Architecture: x86_64
- CPU op-mode(s): 32-bit, 64-bit
- Byte Order: Little Endian
- CPU(s): 24
- On-line CPU(s) list: 0-23
- Thread(s) per core: 2
- Core(s) per socket: 6
- Socket(s): 2
- NUMA node(s): 2
- Vendor ID: GenuineIntel
- CPU family: 6
- Model: 63
- Model name: Intel(R) Xeon(R) CPU E5-2640 v3 @ 2.60GHz
- Stepping: 2
- CPU MHz: 2907.656
- BogoMIPS: 4804.69
- Virtualization: VT-x
- L1d cache: 32K

- L1i cache: 32K
- L2 cache: 256K
- L3 cache: 15360K
- NUMA node0 CPU(s): 0-5, 12-17
- NUMA node1 CPU(s): 6-11, 18-23

3 Problem Description

3.1 Linear Interpolation (Scatter)

Given a set of scattered points in a 2D domain with known function values:

$$S = \{(x_i, y_i, f_i) \mid i = 1, 2, \ldots, N\}$$

where (x_i, y_i) represent spatial coordinates and f_i are known function values. For this assignment, assume $f_i = 1$. The objective is to interpolate these scattered values onto a structured mesh grid of size $M \times M$.

The structured mesh consists of regularly spaced grid points:

$$G = \{(X_i, Y_j) \mid X_i = i\Delta x,\ Y_j = j\Delta y,\ i, j = 0, 1, \ldots, M-1\}$$

where Δx and Δy define uniform grid spacing:

$$\Delta x = \frac{X_{\max}}{M}, \quad \Delta y = \frac{Y_{\max}}{M}$$

All scattered points lie within a domain of size 1×1. The goal is to compute interpolated values $F(X_i, Y_j)$ on the structured grid using linear interpolation.

3.2 Particle Update using Reverse Interpolation (Mover)

In addition to mapping particle values to the grid, the workflow introduces a reverse interpolation step (mover). After the grid values are calculated and normalized to a fixed range, they influence the motion of particles.

For a particle located at (x_i, y_i), the field value is gathered using the same four neighboring grid points and identical weights:

$$F_i = w_{i,j}F(X_i, Y_j) + w_{i+1,j}F(X_{i+1}, Y_j) + w_{i,j+1}F(X_i, Y_{j+1}) + w_{i+1,j+1}F(X_{i+1}, Y_{j+1})$$

The particle position is then updated as:

$$x_i^{new} = x_i + F_i \cdot \Delta x$$

$$y_i^{new} = y_i + F_i \cdot \Delta y$$

If a particle moves outside the domain, it is marked as inactive and excluded from further computations.

3.3 Parallelization Strategies:

- Domain decomposition vs. Particle decomposition
- Efficient handling of race conditions during the scatter step (e.g., atomic operations vs privatization)
- Memory access patterns for both grid and particle data
- Proper load balancing as particle distribution evolves
- Techniques to minimize synchronization overhead

We are expected to make a **hybrid (MPI+OpenMP)** code. The MPI parallelization needs to be added to the existing code to include communication across nodes. This MPI layer will enable you to use all the cores available on the HPC.

4 Benchmarking Methodology

Input Data Configurations

Input data is generated using `input_fileMaker.c` with the following configurations:

(a) $N_x = 250$, $N_y = 100$, Number of points $= 0.9 \times 10^6$, Maxiter $= 10$

(b) $N_x = 250$, $N_y = 100$, Number of points $= 5 \times 10^6$, Maxiter $= 10$

(c) $N_x = 500$, $N_y = 200$, Number of points $= 3.6 \times 10^6$, Maxiter $= 10$

(d) $N_x = 500$, $N_y = 200$, Number of points $= 20 \times 10^6$, Maxiter $= 10$

(e) $N_x = 1000$, $N_y = 400$, Number of points $= 14 \times 10^6$, Maxiter $= 10$

5 Graphical Results

5.1 Speedup vs Cores

5.1.1 Config01: $N_x = 250$, $N_y = 100$, Number of points $= 0.9 \times 10^6$, Maxiter $= 10$

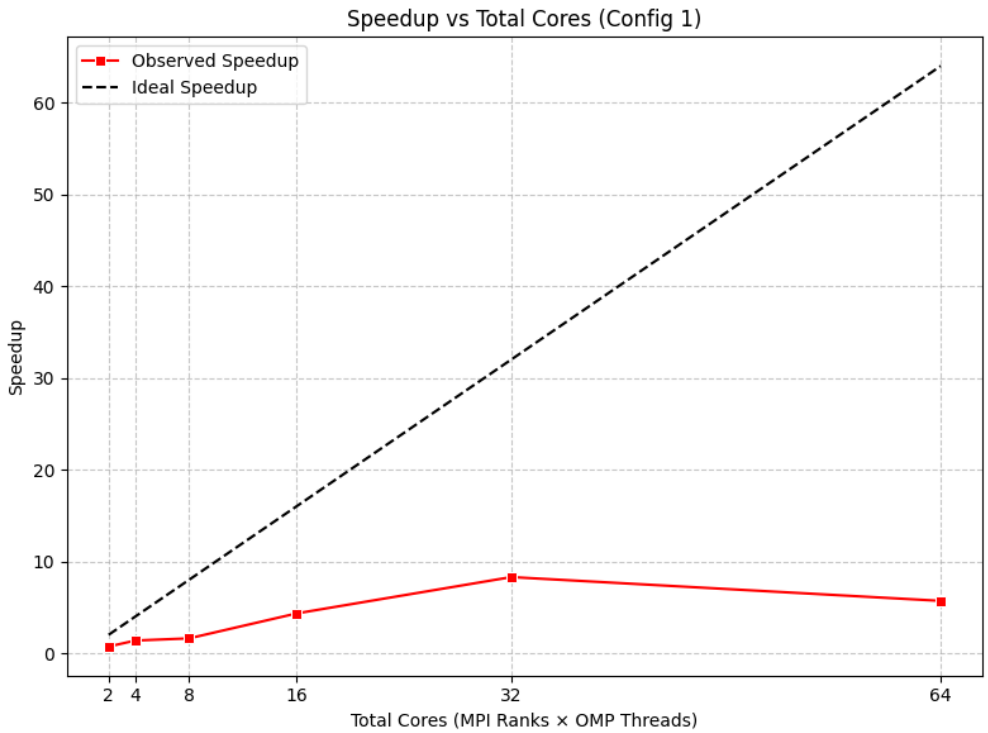


Figure 1: Speedup vs Number of Cores - Config 01

5.1.2 Config02: $N_x = 250$, $N_y = 100$, Number of points $= 5 \times 10^6$, Maxiter $= 10$

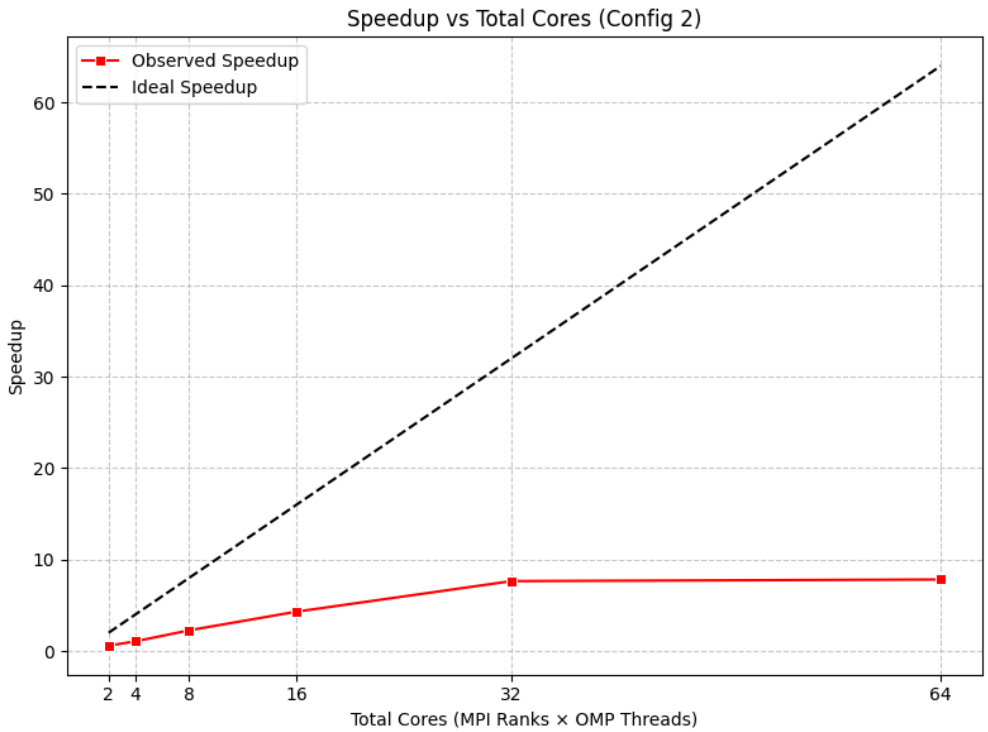


Figure 2: Speedup vs Number of Cores - Config 02

5.1.3 Config03: $N_x = 500$, $N_y = 200$, Number of points $= 3.6 \times 10^6$, Maxiter $= 10$

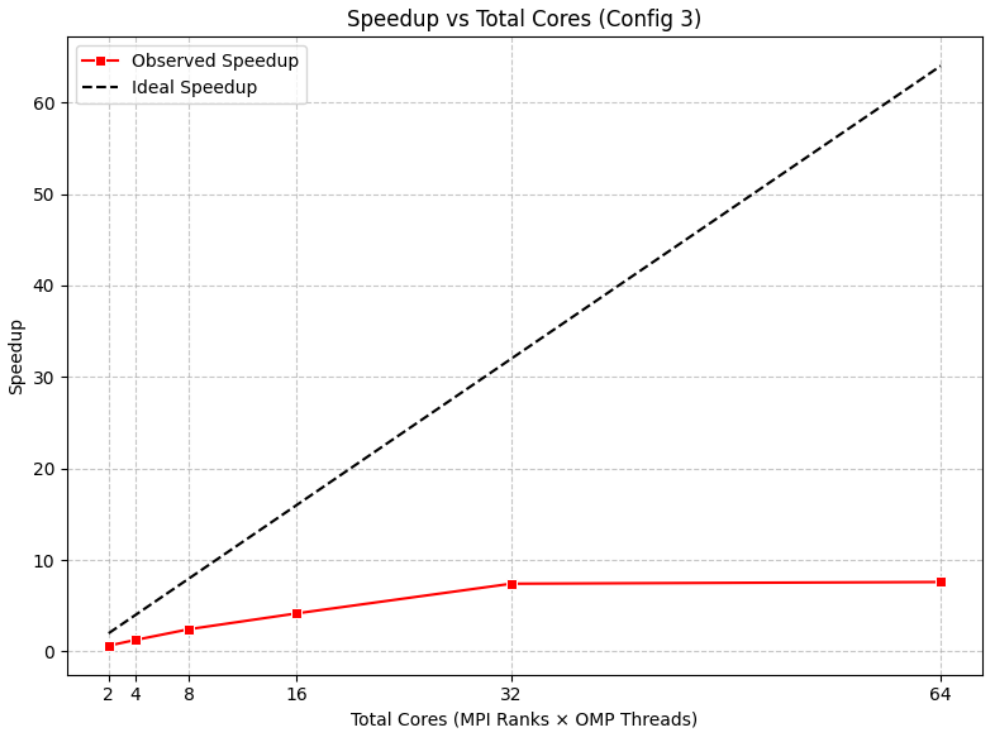


Figure 3: Speedup vs Number of Cores - Config 03

5.1.4 Config04: $N_x = 500$, $N_y = 200$, Number of points $= 20 \times 10^6$, Maxiter $= 10$

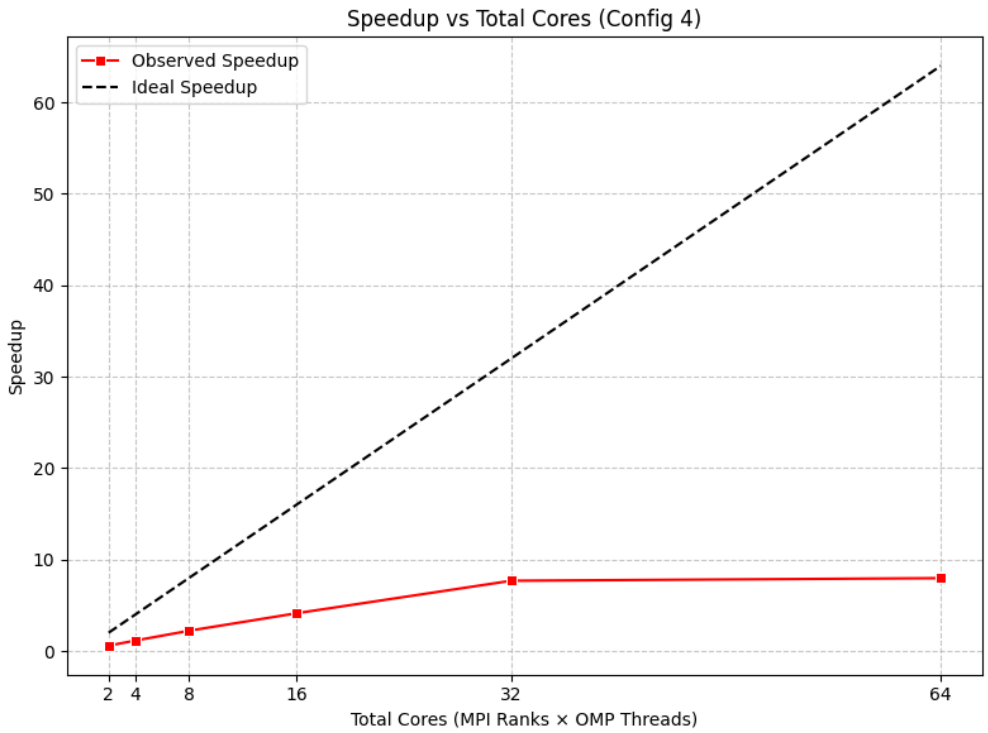


Figure 4: Speedup vs Number of Cores - Config 04

5.1.5 Config05: $N_x = 1000$, $N_y = 400$, Number of points $= 14 \times 10^6$, Maxiter $= 10$

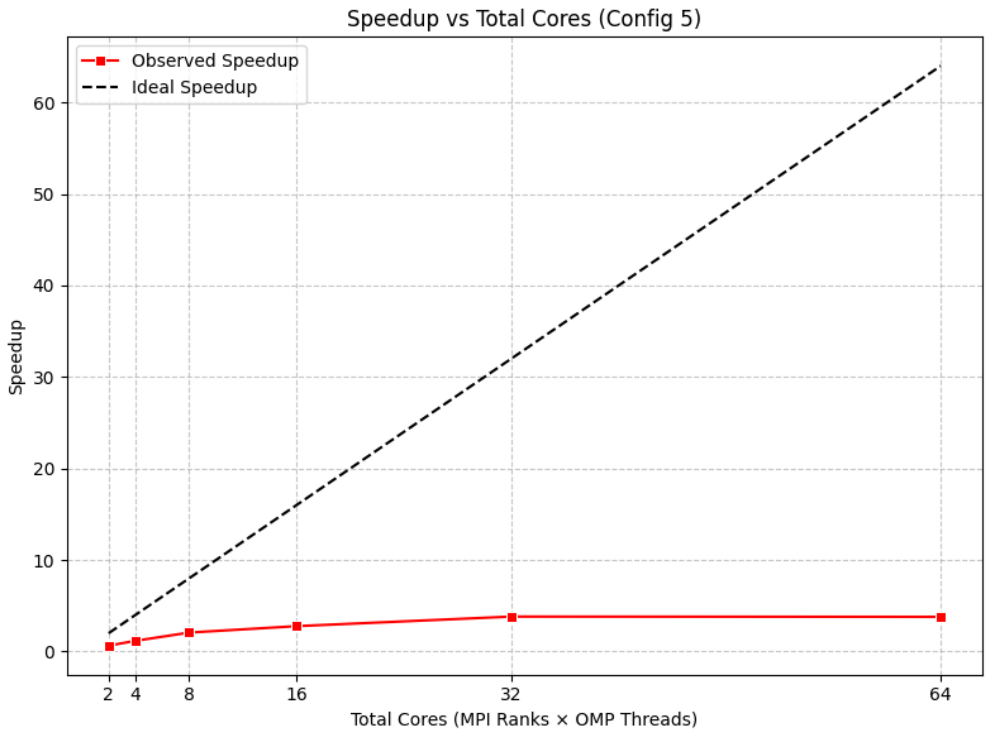


Figure 5: Speedup vs Number of Cores - Config 05

5.2 Execution time vs Cores

5.2.1 Config01: $N_x = 250$, $N_y = 100$, Number of points $= 0.9 \times 10^6$, Maxiter $= 10$

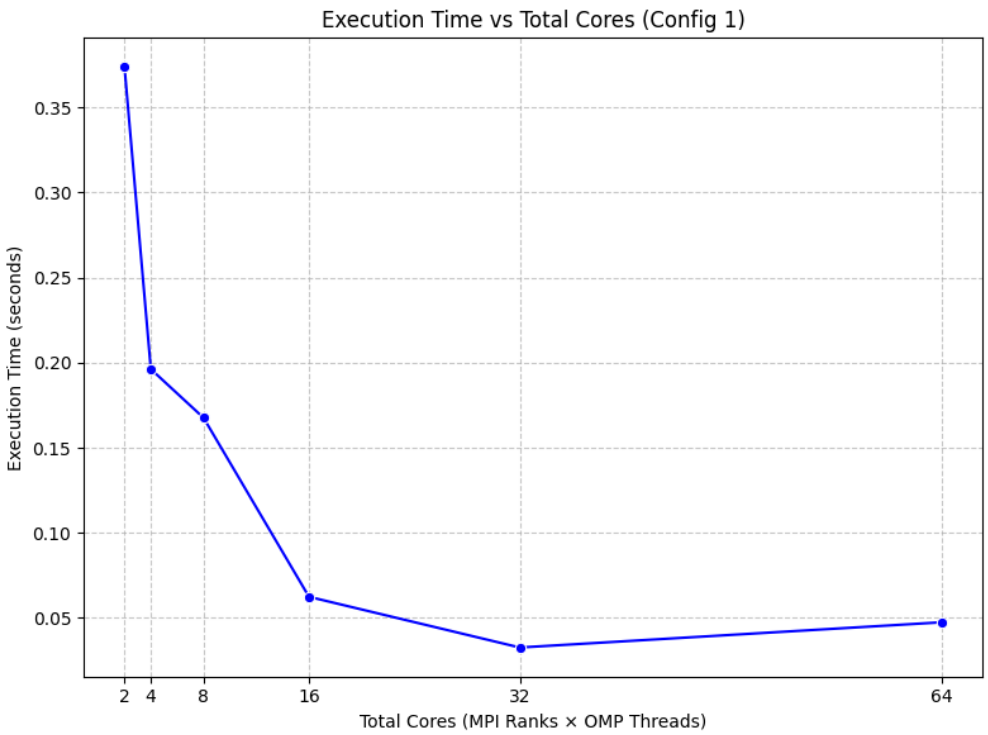


Figure 6: Execution Time vs Number of Cores - Config 01

5.2.2 Config02: $N_x = 250$, $N_y = 100$, Number of points $= 5 \times 10^6$, Maxiter $= 10$

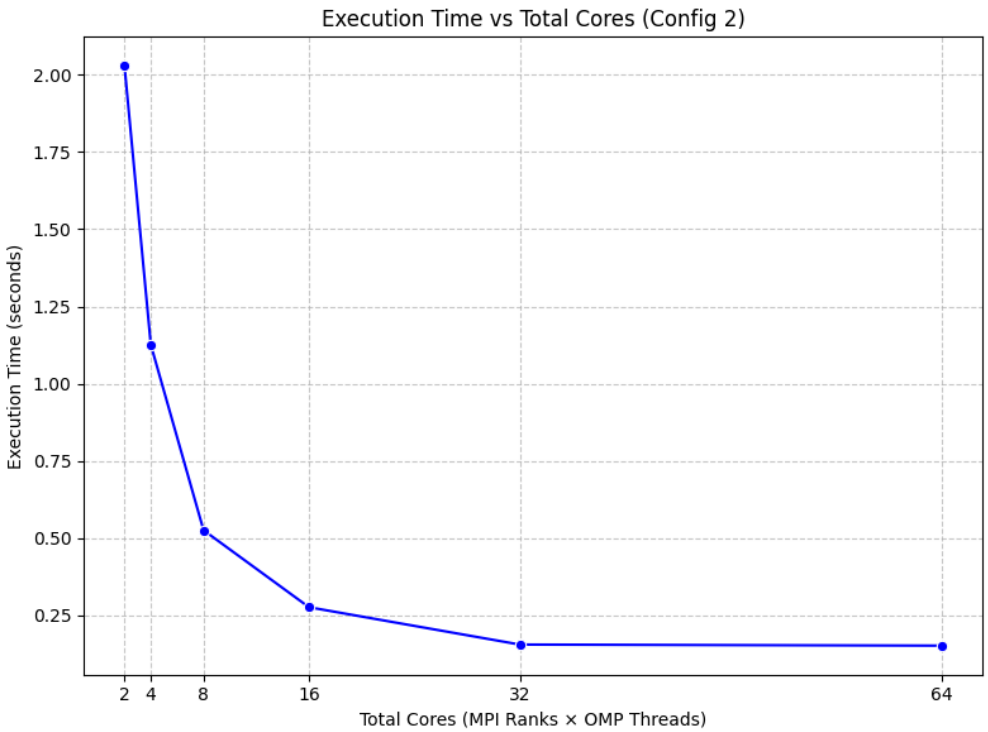


Figure 7: Execution Time vs Number of Cores - Config 02

5.2.3 Config03: $N_x = 500$, $N_y = 200$, Number of points $= 3.6 \times 10^6$, Maxiter $= 10$

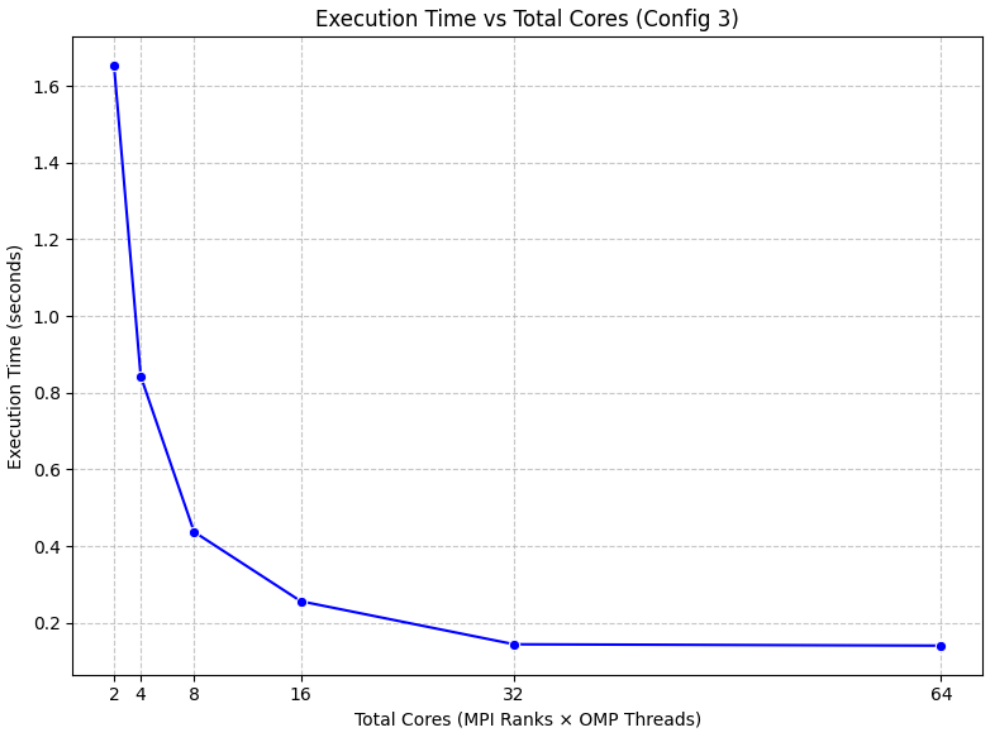


Figure 8: Execution Time vs Number of Cores - Config 03

5.2.4 Config04: $N_x = 500$, $N_y = 200$, Number of points $= 20 \times 10^6$, Maxiter $= 10$

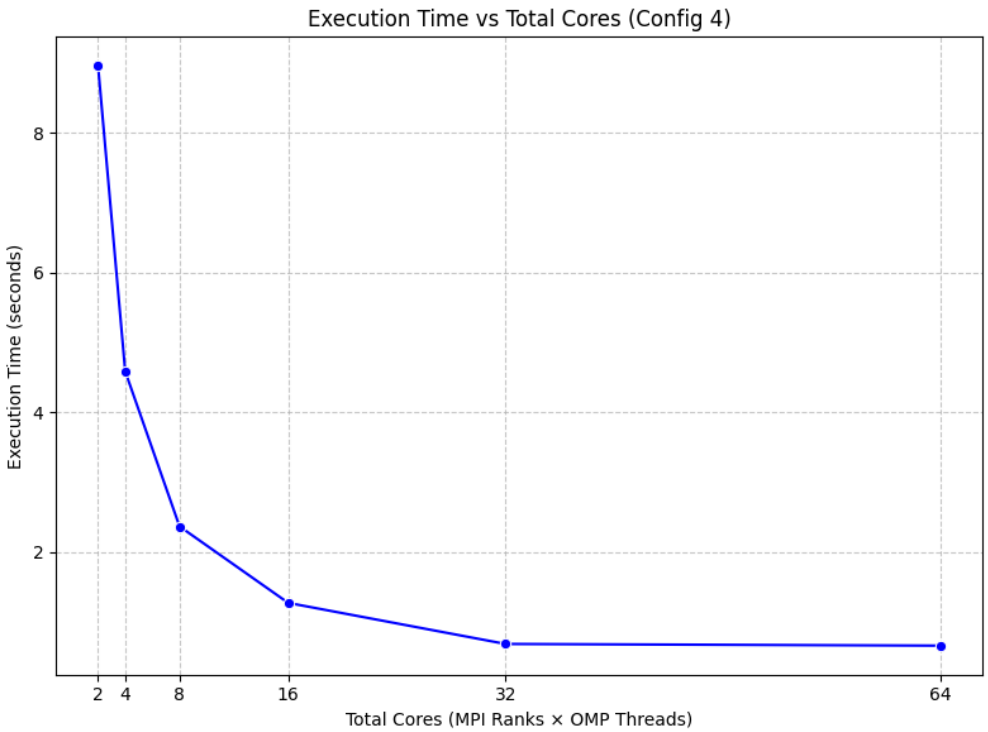


Figure 9: Execution Time vs Number of Cores - Config 04

5.2.5 Config05: $N_x = 1000$, $N_y = 400$, Number of points $= 14 \times 10^6$, Maxiter $= 10$

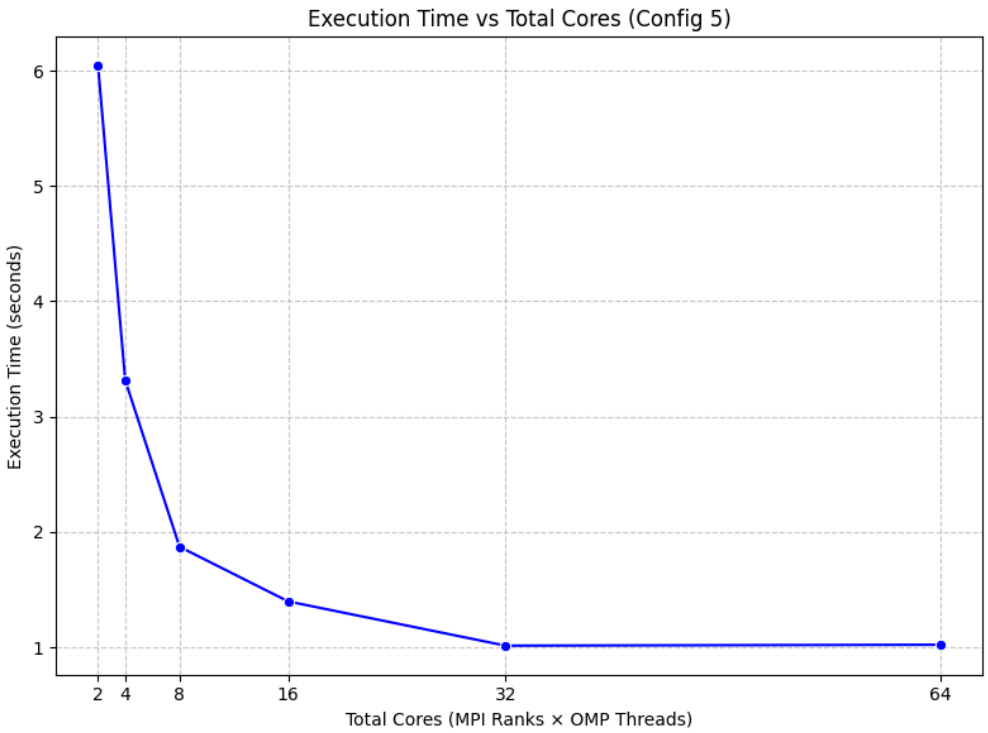


Figure 10: Execution Time vs Number of Cores - Config 05

6 Analysis and Questions

Q1. **Write the pseudocode of your parallel implementation in the report along with an illustrative diagram of your implementation.**

(i) Pseudocode: Hybrid MPI+OpenMP Pipeline

Algorithm 1 Hybrid Parallel Interpolation and Mover (MPI + OpenMP)

Input: mesh_value array, points array
Output: Updated particle positions and final mesh

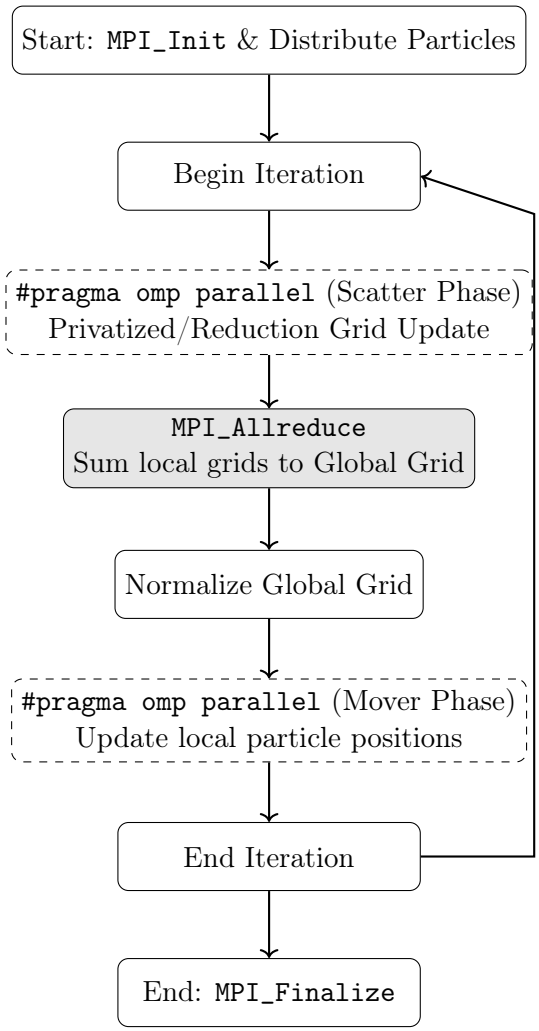
```
Initialize MPI_Init() and get rank, size
 Distribute N particles evenly across MPI ranks (Local_N = N/size)

for iter = 1 to Maxiter do
    Reset local_mesh to 0

    // SCATTER PHASE: Grid Interpolation
     #pragma omp parallel for reduction(+:local_mesh) // Privatization
     for each particle p = 1 to Local_N do in parallel
        if points[p].is_void then
            continue
        Calculate weights (w00, w10, w01, w11)
         Update local_mesh based on weights
    // INTER-NODE COMMUNICATION
     MPI_Allreduce(local_mesh, global_mesh, MPI_SUM)
     Normalize global_mesh to [-1, 1]

    // GATHER PHASE: Particle Mover
     #pragma omp parallel for
     for each particle p = 1 to Local_N do in parallel
        if points[p].is_void then
            continue
        Read weights and gather F from global_mesh
         x ← x + F · dx,   y ← y + F · dy
         if out of bounds then
            points[p].is_void ← true
MPI_Finalize()
```

(ii) Schematic Flow Diagram: Hybrid Pipeline



Q2. **Explain your parallel implementation approach and how you handled race conditions in your code.**

- **Hybrid Decomposition Strategy:**
  (a) The implementation uses **particle decomposition** distributed across both MPI processes and OpenMP threads.
  (b) At the cluster level, MPI divides the total dataset of particles equally among the compute nodes.
  (c) At the node level, OpenMP parallelizes the computation for that chunk of particles across the available CPU cores.
- **Handling Race Conditions:**
  (a) **Scatter Phase:** Since multiple particles handled by different OpenMP threads can map to the same grid points, race conditions occur locally. We handle this using grid

reduction or thread-privatization, creating a local grid copy per thread to accumulate values without locks.

(b) **Inter-node Communication:** Because boundaries are shared across nodes, `MPI_Allreduce` is used to safely sum all local sub-grids into a single synchronized global grid.

(c) **Mover Phase:** The shared grid is accessed purely as read-only, and each particle updates its own private position memory. This phase operates seamlessly without atomics or critical sections.

Q3. **Plot graphs for each of the five input configurations for speedup vs cores and execution time vs cores (increment cores starting from 2 to 64 by a factor of 2). Explain the difference between a login node and a compute node.**

Refer to the Graphical Results section above for the plots.

Login vs. Compute Node: A **login node** serves as the entry point into an HPC cluster. It is intended for lightweight tasks like editing code, compiling software, managing files, and submitting jobs to the scheduler. A **compute node** is a dedicated machine within the cluster architecture designed explicitly to execute computationally heavy, parallel jobs. Compute nodes (like `gies1-4`) run the actual workload without interference from background user interactions.

Q4. **Compute the parallel efficiency and comment on the scalability of your implementation.**

Using Config 4 as an example, parallel efficiency drops from ∼29% (2 cores) to ∼12% (64 cores). The implementation scales reasonably well up to 32 cores, but hits strong scaling limitations at 64 cores where execution time plateaus due to dominating synchronization and communication overheads.

Q5. **Compare the performance of your parallel code with the serial version and report the maximum speedup achieved.**

The maximum speedup achieved is **8.29x** (Config 1 using 32 cores). Across most other configurations, the maximum speedups ranged between 7.5x and 8.0x compared to their respective serial baselines.

Q6. **Try to explain the speedup achieved and all the observations using your current HPC knowledge.**

The sub-linear speedup (e.g., ∼8x on 32-64 cores) is primarily caused by:

- **Memory Wall:** Scatter interpolation creates irregular memory accesses, leading to cache misses that saturate bandwidth.
- **Synchronization Overhead:** Accumulating grid values requires atomic operations or global MPI reductions, causing severe bottlenecks.
- **Amdahl's Law:** Unparallelized sequential portions (like I/O or normalizations) limit the theoretical maximum speedup.

Q7. **If given unlimited HPC resources, how would you further optimize the implementation?**

We can apply three major optimizations:

- **GPU Acceleration (CUDA/OpenACC):** To leverage massive parallel threads and high memory bandwidth.
- **Particle Sorting:** Binning particles by spatial location to ensure contiguous, cache-friendly memory accesses during interpolation.
- **Spatial Domain Decomposition:** Dividing the grid among MPI ranks and using halo exchanges (ghost zones) to drastically reduce global communication overhead.

Q8. **Discuss any load imbalance or performance bottlenecks observed in your implementation.**

- Load Imbalance due to Inactive Particles: Over time, particles move outside the domain and become inactive, creating a load imbalance across threads and MPI ranks that hold regions with more void particles.
- Communication Overhead: The use of `MPI_Allreduce` creates a synchronization barrier across all nodes during every iteration, representing a bottleneck as we scale to 64 cores.
- Memory Latency: The gather/mover operation is highly read-intensive with non-contiguous grid access, leading to cache misses.

Q9. **Suggest at least one alternative approach (data structure or parallelization method) to improve performance.**

Instead of particle decomposition, **Domain Decomposition** combined with ghost cells could be implemented. In this approach, each MPI rank owns a specific spatial region of the grid and only the particles within it. While this adds overhead to migrate particles between nodes when they cross boundaries, it removes the need for a costly global `MPI_Allreduce` on the entire grid, swapping it for cheaper halo/ghost-cell point-to-point exchanges.

Q10. **You are required to develop alternative data structures or OpenMP-MPI-based approaches to improve the code's performance.**

- **Structure of Arrays (SoA):** To optimize the OpenMP-MPI workflow, converting the particle data structure from Array of Structures (AoS: 'x', 'y', 'f', 'is_void') to Structure of Arrays (SoA: array of 'x's, array of 'y's) allows for better SIMD vectorization and cache locality during the read-heavy mover phase.
- **Dynamic OpenMP Scheduling:** Applying dynamic loop scheduling inside the MPI ranks helps smooth out intra-node load imbalances caused by void particles.

Q11. **Analyze the performance of interpolation and mover phases separately. Which phase is the bottleneck and why?**

The **Interpolation phase** is the bottleneck, consuming ∼55–60% of the total execution time compared to the Mover phase (∼40–45%). While the mover phase is "embarrassingly parallel" with independent updates, the interpolation phase requires multiple particles to write to shared grid points. This causes race conditions, requiring expensive atomic operations and synchronization that thrash the cache and slow down execution.

7 Conclusions

- **Phase bottlenecks:** The interpolation phase is the primary bottleneck ($\sim$55–60% of execution time) due to irregular memory access and heavy synchronization overhead required to prevent race conditions.
- **Parallel performance:** The hybrid MPI+OpenMP implementation achieved a maximum speedup of 8.29$\times$. However, scalability degrades severely at 64 cores (dropping to $\sim$12% efficiency) as communication costs outpace computation.
- **Hybrid Parallelization impact:** Distributed execution exposes a "memory wall." The high cost of global MPI reductions across nodes demonstrates that adding more cores cannot overcome inherent communication bottlenecks.
- **Future optimizations:** To achieve better scaling, future iterations must adopt spatial domain decomposition (using ghost/halo zones), particle sorting for cache locality, or GPU offloading to leverage higher memory bandwidth.